MasterNotebook

November 29, 2025

1 Master Notebook

1.1 Dataset Overview and Motivation

1.1.1 TCGA-OV Cohort

The TCGA Ovarian Serous Custadenocarcinoma (TCGA-OV) cohort is one of the most comprehensive datasets available for molecular, genetic expression, clinical and phenotypes at patient level. The data set includes multi-omic assays curated through the NCI Genomic Data Commons (GDC).

These datasets were downloaded using the UCSC Xena Browser (associated download scripts, as described in the project README.)

Generating high-fidelity synthetic biomedical data serves many practical and scientific motivations:

(1) Privacy Preservation of data sharing
(2) Augment smaller sample biomedical data
(3) Hypothesis exploration (gene/gene correlation structures etc.)

The TCGA-OV dataset is well suited to perform Synthetic Data Generation (SDGE), with rich multi-omic hetoerogentiy and good quality clinical meta data.

This notebook explores the following workflow of SDGE on the TCGA-OV dataset:

- preprocessing of data
- contstucting a model-ready feature matrix
- training diffusion-based generative model
- evaluating and comparision of synthetic data

Generated data will be tested for statistical fidelity, correlation and similarity metrics to original data.

[67]:
```
# Importing necessary libraries
import json
import os
import math
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import torch
```

```
import torch.nn as nn
from scipy.spatial.distance import cdist
from scipy.stats import ks_2samp
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, Dataset
import torch.nn.functional as F
from scipy.spatial.distance import jensenshannon



import os
import json
import math
import joblib
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from scipy.stats import ks_2samp



import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ks_2samp
```

1.2 Data Preprocessing and Exploratory Data Analysis

1.2.1 Gene Expression Dataset Overview

Gene Expression RNA-seq – STAR – FPKM is a gene-by-sample matrix representing normalized gene expression levels for each TCGA tumor sample.

[68]:
```
# TCGA-OV Gene expression DF
gene_expression = pd.read_csv("data/raw/TCGA-OV.star_fpkm.tsv.gz", sep="\t",␣
↪index_col=0)
```

[69]:
```
gene_expression.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 60660 entries, ENSG00000000003.15 to ENSG00000288675.1
Columns: 429 entries, TCGA-24-1104-01A to TCGA-13-0890-01A
dtypes: float64(429)
memory usage: 199.0+ MB
```

[70]:
```
gene_expression.head(5)
```

[70]:

Ensembl_ID	TCGA-24-1104-01A	TCGA-25-1320-01A	TCGA-24-1850-01A	\
ENSG00000000003.15	2.983714	4.479334	3.717309	
ENSG00000000005.6	0.017494	0.027720	0.338111	
ENSG00000000419.13	5.456290	5.600154	5.362968	
ENSG00000000457.14	1.824157	2.038155	1.322390	
ENSG00000000460.17	0.802814	2.139764	1.632408	

Ensembl_ID	TCGA-61-2101-01A	TCGA-23-1111-01A	TCGA-25-2396-01A	\
ENSG00000000003.15	3.480847	4.319249	4.329037	
ENSG00000000005.6	0.022616	0.014212	0.090447	
ENSG00000000419.13	5.440248	6.086251	5.811286	
ENSG00000000457.14	1.505332	1.323543	1.426426	
ENSG00000000460.17	1.375735	1.332221	1.396324	

Ensembl_ID	TCGA-30-1714-01A	TCGA-04-1347-01A	TCGA-59-2351-01A	\
ENSG00000000003.15	4.298101	4.778608	3.589967	
ENSG00000000005.6	0.066537	0.255078	0.019061	
ENSG00000000419.13	5.901470	6.521613	5.392046	
ENSG00000000457.14	1.643902	1.269273	1.672335	
ENSG00000000460.17	1.220392	0.828875	1.643625	

Ensembl_ID	TCGA-09-1659-01B	…	TCGA-25-1632-01A	TCGA-25-2404-01A	\
		…			
ENSG00000000003.15	3.887730	…	4.559639	3.716826	
ENSG00000000005.6	0.067226	…	0.015212	0.343806	
ENSG00000000419.13	5.832110	…	5.127720	5.389536	
ENSG00000000457.14	1.371726	…	1.336512	1.348006	
ENSG00000000460.17	1.149129	…	1.032242	1.238175	

Ensembl_ID	TCGA-25-1323-01A	TCGA-13-0886-01A	TCGA-24-2262-01A	\
ENSG00000000003.15	4.561179	5.248083	3.554147	
ENSG00000000005.6	0.344033	0.849439	0.083111	
ENSG00000000419.13	5.956705	5.948577	4.922345	
ENSG00000000457.14	1.629473	1.229834	1.716903	
ENSG00000000460.17	1.710349	0.859015	1.567107	

```
                    TCGA-36-1568-01A  TCGA-20-1685-01A  TCGA-61-1738-01A  \
Ensembl_ID
ENSG00000000003.15          3.849489          4.986411          4.634535
ENSG00000000005.6           0.029842          0.038858          0.383386
ENSG00000000419.13          5.195344          5.412148          5.429743
ENSG00000000457.14          1.547697          1.383055          1.466079
ENSG00000000460.17          1.072037          1.507921          1.115499

                    TCGA-25-1623-01A  TCGA-13-0890-01A
Ensembl_ID
ENSG00000000003.15          3.303489          4.518869
ENSG00000000005.6           0.092072          0.436375
ENSG00000000419.13          6.149123          4.910944
ENSG00000000457.14          1.377013          0.898324
ENSG00000000460.17          1.411209          1.229772

[5 rows x 429 columns]
```

[71]:
```
gene_expression.isna().sum().sort_values(ascending=False)
```

[71]:
```
TCGA-24-1104-01A    0
TCGA-29-1701-01A    0
TCGA-13-1499-01A    0
TCGA-29-1766-01A    0
TCGA-13-0801-01A    0
                   ..
TCGA-25-1871-01A    0
TCGA-24-2289-01A    0
TCGA-24-1562-01A    0
TCGA-24-1557-01A    0
TCGA-13-0890-01A    0
Length: 429, dtype: int64
```

There are no missing values in the **gene expression** dataset.

1.2.2 Phenotype/Clinical Data Overview

The clinical annotation table contains patient level variables: survival, tumor stage, treatment, and demographic information

[72]:
```
# TCGA-OV clinical/phenotypic DF
phenotype = pd.read_csv('data/raw/TCGA-OV.clinical.tsv.gz', sep="\t",␣
 ↪index_col=0)
```

[73]:
```
phenotype.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 632 entries, TCGA-13-0920-01A to TCGA-61-1899-01A
```

```
Data columns (total 83 columns):
 #   Column                                                Non-Null Count
Dtype
---  ------                                                --------------
-----
 0   id                                                    632 non-null
object
 1   disease_type                                          632 non-null
object
 2   case_id                                               632 non-null
object
 3   submitter_id                                          632 non-null
object
 4   primary_site                                          632 non-null
object
 5   alcohol_history.exposures                             613 non-null
object
 6   race.demographic                                      613 non-null
object
 7   gender.demographic                                    613 non-null
object
 8   ethnicity.demographic                                 613 non-null
object
 9   vital_status.demographic                              613 non-null
object
 10  age_at_index.demographic                              613 non-null
float64
 11  days_to_birth.demographic                             602 non-null
float64
 12  year_of_birth.demographic                             613 non-null
float64
 13  days_to_death.demographic                             365 non-null
float64
 14  year_of_death.demographic                             315 non-null
float64
 15  primary_site.project                                  632 non-null
object
 16  project_id.project                                    632 non-null
object
 17  disease_type.project                                  632 non-null
object
 18  name.project                                          632 non-null
object
 19  name.program.project                                  632 non-null
object
 20  tissue_source_site_id.tissue_source_site              632 non-null
object
 21  code.tissue_source_site                               632 non-null
```

```
object
 22  name.tissue_source_site                                632 non-null
object
 23  project.tissue_source_site                             632 non-null
object
 24  bcr_id.tissue_source_site                              632 non-null
object
 25  entity_submitter_id.annotations                        62 non-null
object
 26  notes.annotations                                      62 non-null
object
 27  submitter_id.annotations                               62 non-null
object
 28  classification.annotations                             62 non-null
object
 29  entity_id.annotations                                  62 non-null
object
 30  created_datetime.annotations                           62 non-null
object
 31  annotation_id.annotations                              62 non-null
object
 32  entity_type.annotations                                62 non-null
object
 33  updated_datetime.annotations                           62 non-null
object
 34  case_id.annotations                                    62 non-null
object
 35  state.annotations                                      62 non-null
object
 36  category.annotations                                   62 non-null
object
 37  status.annotations                                     62 non-null
object
 38  case_submitter_id.annotations                          62 non-null
object
 39  figo_stage.diagnoses                                   608 non-null
object
 40  synchronous_malignancy.diagnoses                       613 non-null
object
 41  days_to_diagnosis.diagnoses                            613 non-null
float64
 42  last_known_disease_status.diagnoses                    613 non-null
object
 43  tissue_or_organ_of_origin.diagnoses                    613 non-null
object
 44  days_to_last_follow_up.diagnoses                       543 non-null
float64
 45  age_at_diagnosis.diagnoses                             602 non-null
```

```
float64
 46  primary_diagnosis.diagnoses                                  613 non-null
object
 47  prior_malignancy.diagnoses                                   613 non-null
object
 48  year_of_diagnosis.diagnoses                                  613 non-null
float64
 49  prior_treatment.diagnoses                                    613 non-null
object
 50  morphology.diagnoses                                         613 non-null
object
 51  classification_of_tumor.diagnoses                            613 non-null
object
 52  icd_10_code.diagnoses                                        613 non-null
object
 53  site_of_resection_or_biopsy.diagnoses                        613 non-null
object
 54  tumor_grade.diagnoses                                        613 non-null
object
 55  progression_or_recurrence.diagnoses                          613 non-null
object
 56  age_at_earliest_diagnosis.diagnoses.xena_derived             602 non-null
float64
 57  age_at_earliest_diagnosis_in_years.diagnoses.xena_derived    602 non-null
float64
 58  updated_datetime.treatments.diagnoses                        613 non-null
object
 59  treatment_id.treatments.diagnoses                            613 non-null
object
 60  submitter_id.treatments.diagnoses                            613 non-null
object
 61  treatment_type.treatments.diagnoses                          613 non-null
object
 62  state.treatments.diagnoses                                   613 non-null
object
 63  treatment_or_therapy.treatments.diagnoses                    613 non-null
object
 64  created_datetime.treatments.diagnoses                        613 non-null
object
 65  sample_type_id.samples                                       632 non-null
int64
 66  tumor_descriptor.samples                                     632 non-null
object
 67  sample_id.samples                                            632 non-null
object
 68  sample_type.samples                                          632 non-null
object
 69  composition.samples                                          347 non-null
```

```
object
 70  days_to_collection.samples                                  10 non-null
float64
 71  initial_weight.samples                                      10 non-null
float64
 72  preservation_method.samples                                 632 non-null
object
 73  intermediate_dimension.samples                              612 non-null
float64
 74  pathology_report_uuid.samples                               610 non-null
object
 75  shortest_dimension.samples                                  612 non-null
float64
 76  oct_embedded.samples                                        10 non-null
object
 77  specimen_type.samples                                       632 non-null
object
 78  days_to_sample_procurement.samples                          1 non-null
float64
 79  longest_dimension.samples                                   612 non-null
float64
 80  is_ffpe.samples                                             632 non-null
bool
 81  tissue_type.samples                                         632 non-null
object
 82  annotations.samples                                         2 non-null
object
dtypes: bool(1), float64(17), int64(1), object(64)
memory usage: 410.4+ KB
```

[74]:
```
phenotype.columns
```

[74]:
```
Index(['id', 'disease_type', 'case_id', 'submitter_id', 'primary_site',
       'alcohol_history.exposures', 'race.demographic', 'gender.demographic',
       'ethnicity.demographic', 'vital_status.demographic',
       'age_at_index.demographic', 'days_to_birth.demographic',
       'year_of_birth.demographic', 'days_to_death.demographic',
       'year_of_death.demographic', 'primary_site.project',
       'project_id.project', 'disease_type.project', 'name.project',
       'name.program.project', 'tissue_source_site_id.tissue_source_site',
       'code.tissue_source_site', 'name.tissue_source_site',
       'project.tissue_source_site', 'bcr_id.tissue_source_site',
       'entity_submitter_id.annotations', 'notes.annotations',
       'submitter_id.annotations', 'classification.annotations',
       'entity_id.annotations', 'created_datetime.annotations',
       'annotation_id.annotations', 'entity_type.annotations',
       'updated_datetime.annotations', 'case_id.annotations',
```

```
       'state.annotations', 'category.annotations', 'status.annotations',
       'case_submitter_id.annotations', 'figo_stage.diagnoses',
       'synchronous_malignancy.diagnoses', 'days_to_diagnosis.diagnoses',
       'last_known_disease_status.diagnoses',
       'tissue_or_organ_of_origin.diagnoses',
       'days_to_last_follow_up.diagnoses', 'age_at_diagnosis.diagnoses',
       'primary_diagnosis.diagnoses', 'prior_malignancy.diagnoses',
       'year_of_diagnosis.diagnoses', 'prior_treatment.diagnoses',
       'morphology.diagnoses', 'classification_of_tumor.diagnoses',
       'icd_10_code.diagnoses', 'site_of_resection_or_biopsy.diagnoses',
       'tumor_grade.diagnoses', 'progression_or_recurrence.diagnoses',
       'age_at_earliest_diagnosis.diagnoses.xena_derived',
       'age_at_earliest_diagnosis_in_years.diagnoses.xena_derived',
       'updated_datetime.treatments.diagnoses',
       'treatment_id.treatments.diagnoses',
       'submitter_id.treatments.diagnoses',
       'treatment_type.treatments.diagnoses', 'state.treatments.diagnoses',
       'treatment_or_therapy.treatments.diagnoses',
       'created_datetime.treatments.diagnoses', 'sample_type_id.samples',
       'tumor_descriptor.samples', 'sample_id.samples', 'sample_type.samples',
       'composition.samples', 'days_to_collection.samples',
       'initial_weight.samples', 'preservation_method.samples',
       'intermediate_dimension.samples', 'pathology_report_uuid.samples',
       'shortest_dimension.samples', 'oct_embedded.samples',
       'specimen_type.samples', 'days_to_sample_procurement.samples',
       'longest_dimension.samples', 'is_ffpe.samples', 'tissue_type.samples',
       'annotations.samples'],
      dtype='object')
```

Note: Some categories of features are `.samples` `.annotations`, `.project` and others which contain metadata.

[75]:
```
phenotype.head(5)
```

[75]:
```
                                                    id  \
sample
TCGA-13-0920-01A  85a85a11-7200-4e96-97af-6ba26d680d59
TCGA-42-2582-01A  7922df77-f09a-488c-a1be-58646ceb9b3e
TCGA-04-1342-01A  8727855e-120a-4216-a803-8cc6cd1159be
TCGA-13-1819-02A  82e96c6c-a88c-4e52-be56-7f24f6c7b835
TCGA-13-1819-01A  82e96c6c-a88c-4e52-be56-7f24f6c7b835

                                              disease_type  \
sample
TCGA-13-0920-01A  Cystic, Mucinous and Serous Neoplasms
TCGA-42-2582-01A  Cystic, Mucinous and Serous Neoplasms
TCGA-04-1342-01A  Cystic, Mucinous and Serous Neoplasms
TCGA-13-1819-02A  Cystic, Mucinous and Serous Neoplasms
```

```
TCGA-13-1819-01A  Cystic, Mucinous and Serous Neoplasms

                                              case_id  submitter_id  \
sample
TCGA-13-0920-01A  85a85a11-7200-4e96-97af-6ba26d680d59  TCGA-13-0920
TCGA-42-2582-01A  7922df77-f09a-488c-a1be-58646ceb9b3e  TCGA-42-2582
TCGA-04-1342-01A  8727855e-120a-4216-a803-8cc6cd1159be  TCGA-04-1342
TCGA-13-1819-02A  82e96c6c-a88c-4e52-be56-7f24f6c7b835  TCGA-13-1819
TCGA-13-1819-01A  82e96c6c-a88c-4e52-be56-7f24f6c7b835  TCGA-13-1819

                primary_site alcohol_history.exposures race.demographic  \
sample
TCGA-13-0920-01A       Ovary              Not Reported            white
TCGA-42-2582-01A       Ovary              Not Reported            white
TCGA-04-1342-01A       Ovary              Not Reported            white
TCGA-13-1819-02A       Ovary              Not Reported            white
TCGA-13-1819-01A       Ovary              Not Reported            white

                gender.demographic   ethnicity.demographic  \
sample
TCGA-13-0920-01A             female  not hispanic or latino
TCGA-42-2582-01A             female  not hispanic or latino
TCGA-04-1342-01A             female            not reported
TCGA-13-1819-02A             female            not reported
TCGA-13-1819-01A             female            not reported

                vital_status.demographic  …  \
sample                                    …
TCGA-13-0920-01A                    Dead  …
TCGA-42-2582-01A                    Dead  …
TCGA-04-1342-01A                    Dead  …
TCGA-13-1819-02A                    Dead  …
TCGA-13-1819-01A                    Dead  …

                 intermediate_dimension.samples  \
sample
TCGA-13-0920-01A                             0.7
TCGA-42-2582-01A                             0.8
TCGA-04-1342-01A                             1.0
TCGA-13-1819-02A                             2.0
TCGA-13-1819-01A                             1.2

                       pathology_report_uuid.samples  \
sample
TCGA-13-0920-01A  6880B406-08B2-43CE-9274-5826B1BE9114
TCGA-42-2582-01A  04201F3C-F4FF-4F85-8C14-F1D754DAFDD6
TCGA-04-1342-01A  B296EAFA-9EE4-4611-888A-FA0B58EC2A88
```

```
TCGA-13-1819-02A  EABB4610-CCE8-4DC5-A416-0FABFA634056
TCGA-13-1819-01A  27225767-34C2-48F2-A347-867FDDC907E9

                  shortest_dimension.samples  oct_embedded.samples  \
sample
TCGA-13-0920-01A                         0.4                   NaN
TCGA-42-2582-01A                         0.2                   NaN
TCGA-04-1342-01A                         0.2                   NaN
TCGA-13-1819-02A                         0.3                   NaN
TCGA-13-1819-01A                         0.4                   NaN

                  specimen_type.samples days_to_sample_procurement.samples  \
sample
TCGA-13-0920-01A               Unknown                                  NaN
TCGA-42-2582-01A               Unknown                                  NaN
TCGA-04-1342-01A               Unknown                                  NaN
TCGA-13-1819-02A          Solid Tissue                                  NaN
TCGA-13-1819-01A          Solid Tissue                                  NaN

                 longest_dimension.samples is_ffpe.samples  \
sample
TCGA-13-0920-01A                       1.0           False
TCGA-42-2582-01A                       1.3           False
TCGA-04-1342-01A                       2.0           False
TCGA-13-1819-02A                       2.1           False
TCGA-13-1819-01A                       1.5           False

                 tissue_type.samples annotations.samples
sample
TCGA-13-0920-01A               Tumor                  NaN
TCGA-42-2582-01A               Tumor                  NaN
TCGA-04-1342-01A               Tumor                  NaN
TCGA-13-1819-02A               Tumor                  NaN
TCGA-13-1819-01A               Tumor                  NaN

[5 rows x 83 columns]
```

[76]:
```
phenotype.select_dtypes('number').describe().T
```

[76]:
```
                                          count          mean  \
age_at_index.demographic                  613.0     59.517129
days_to_birth.demographic                 602.0 -21921.207641
year_of_birth.demographic                 613.0   1944.205546
days_to_death.demographic                 365.0   1170.758904
year_of_death.demographic                 315.0   2003.774603
days_to_diagnosis.diagnoses               613.0      0.000000
days_to_last_follow_up.diagnoses          543.0   1110.755064
```

```
age_at_diagnosis.diagnoses                              602.0  21921.207641
year_of_diagnosis.diagnoses                             613.0   2003.722675
age_at_earliest_diagnosis.diagnoses.xena_derived        602.0  21921.207641
age_at_earliest_diagnosis_in_years.diagnoses.xe…  602.0     60.058103
sample_type_id.samples                                  632.0      1.219937
days_to_collection.samples                               10.0    489.200000
initial_weight.samples                                   10.0    372.000000
intermediate_dimension.samples                          612.0      0.872222
shortest_dimension.samples                              612.0      0.421487
days_to_sample_procurement.samples                        1.0      0.000000
longest_dimension.samples                               612.0      1.366340

                                                          std           min  \
age_at_index.demographic                                11.459154     26.000000
days_to_birth.demographic                             4180.659838 -32853.000000
year_of_birth.demographic                               12.494057   1908.000000
days_to_death.demographic                              778.425123      8.000000
year_of_death.demographic                                3.439651   1992.000000
days_to_diagnosis.diagnoses                              0.000000      0.000000
days_to_last_follow_up.diagnoses                       938.332316      0.000000
age_at_diagnosis.diagnoses                            4180.659838   9760.000000
year_of_diagnosis.diagnoses                              4.070228   1992.000000
age_at_earliest_diagnosis.diagnoses.xena_derived      4180.659838   9760.000000
age_at_earliest_diagnosis_in_years.diagnoses.xe…    11.453863     26.739726
sample_type_id.samples                                   1.372372      1.000000
days_to_collection.samples                             457.927408     11.000000
initial_weight.samples                                 278.240503    130.000000
intermediate_dimension.samples                           0.373701      0.200000
shortest_dimension.samples                               0.184639      0.050000
days_to_sample_procurement.samples                            NaN      0.000000
longest_dimension.samples                                0.598562      0.300000

                                                            25%  \
age_at_index.demographic                                 51.000000
days_to_birth.demographic                             -25004.250000
year_of_birth.demographic                              1935.000000
days_to_death.demographic                               592.000000
year_of_death.demographic                              2002.000000
days_to_diagnosis.diagnoses                               0.000000
days_to_last_follow_up.diagnoses                        363.000000
age_at_diagnosis.diagnoses                            18760.250000
year_of_diagnosis.diagnoses                            2001.000000
age_at_earliest_diagnosis.diagnoses.xena_derived      18760.250000
age_at_earliest_diagnosis_in_years.diagnoses.xe…     51.397945
sample_type_id.samples                                    1.000000
days_to_collection.samples                              120.500000
initial_weight.samples                                  215.000000
```

```
intermediate_dimension.samples                              0.600000
shortest_dimension.samples                                  0.300000
days_to_sample_procurement.samples                          0.000000
longest_dimension.samples                                   1.000000

                                                                 50%  \
age_at_index.demographic                                   58.000000
days_to_birth.demographic                                -21533.000000
year_of_birth.demographic                                 1945.000000
days_to_death.demographic                                 1078.000000
year_of_death.demographic                                 2004.000000
days_to_diagnosis.diagnoses                                  0.000000
days_to_last_follow_up.diagnoses                           918.000000
age_at_diagnosis.diagnoses                               21533.000000
year_of_diagnosis.diagnoses                               2004.000000
age_at_earliest_diagnosis.diagnoses.xena_derived         21533.000000
age_at_earliest_diagnosis_in_years.diagnoses.xe…            58.994521
sample_type_id.samples                                       1.000000
days_to_collection.samples                                 320.500000
initial_weight.samples                                     245.000000
intermediate_dimension.samples                               0.800000
shortest_dimension.samples                                   0.400000
days_to_sample_procurement.samples                           0.000000
longest_dimension.samples                                    1.300000

                                                                 75%  \
age_at_index.demographic                                   68.000000
days_to_birth.demographic                                -18760.250000
year_of_birth.demographic                                 1954.000000
days_to_death.demographic                                 1579.000000
year_of_death.demographic                                 2006.000000
days_to_diagnosis.diagnoses                                  0.000000
days_to_last_follow_up.diagnoses                          1579.000000
age_at_diagnosis.diagnoses                               25004.250000
year_of_diagnosis.diagnoses                               2007.000000
age_at_earliest_diagnosis.diagnoses.xena_derived         25004.250000
age_at_earliest_diagnosis_in_years.diagnoses.xe…            68.504795
sample_type_id.samples                                       1.000000
days_to_collection.samples                                 779.500000
initial_weight.samples                                     385.000000
intermediate_dimension.samples                               1.000000
shortest_dimension.samples                                   0.500000
days_to_sample_procurement.samples                           0.000000
longest_dimension.samples                                    1.500000

                                                                 max
age_at_index.demographic                                   89.000000
days_to_birth.demographic                                -9760.000000
year_of_birth.demographic                                 1982.000000
days_to_death.demographic                                 4624.000000
year_of_death.demographic                                 2013.000000
days_to_diagnosis.diagnoses                                  0.000000
days_to_last_follow_up.diagnoses                          5481.000000
age_at_diagnosis.diagnoses                               32853.000000
year_of_diagnosis.diagnoses                               2013.000000
age_at_earliest_diagnosis.diagnoses.xena_derived         32853.000000
age_at_earliest_diagnosis_in_years.diagnoses.xe…            90.008219
sample_type_id.samples                                      11.000000
days_to_collection.samples                                1211.000000
initial_weight.samples                                    1040.000000
intermediate_dimension.samples                               3.000000
shortest_dimension.samples                                   1.500000
days_to_sample_procurement.samples                           0.000000
longest_dimension.samples                                    8.000000
```

[77]:
```
phenotype.isna().sum().sort_values(ascending=False)
```

[77]:
```
days_to_sample_procurement.samples    631
annotations.samples                   630
oct_embedded.samples                  622
initial_weight.samples                622
days_to_collection.samples            622
                                      ...
name.tissue_source_site                 0
project.tissue_source_site              0
bcr_id.tissue_source_site               0
disease_type                            0
id                                      0
Length: 83, dtype: int64
```

1.2.3 Creating Seed Dataset

[78]:
```
print(gene_expression.shape)
```

```
(60660, 429)
```

[79]:
```
print(phenotype.shape)
```

```
(632, 83)
```

[80]:
```
gene_exp_T = gene_expression.T # Transpose the gene expression data to have␣
 ↪rows as samples
gene_exp_T.head(5)
```

[80]:

Ensembl_ID	ENSG00000000003.15	ENSG00000000005.6	ENSG00000000419.13	\
TCGA-24-1104-01A	2.983714	0.017494	5.456290	
TCGA-25-1320-01A	4.479334	0.027720	5.600154	
TCGA-24-1850-01A	3.717309	0.338111	5.362968	
TCGA-61-2101-01A	3.480847	0.022616	5.440248	
TCGA-23-1111-01A	4.319249	0.014212	6.086251	

Ensembl_ID	ENSG00000000457.14	ENSG00000000460.17	ENSG00000000938.13	\
TCGA-24-1104-01A	1.824157	0.802814	1.465191	
TCGA-25-1320-01A	2.038155	2.139764	1.686926	
TCGA-24-1850-01A	1.322390	1.632408	1.724738	
TCGA-61-2101-01A	1.505332	1.375735	0.708496	
TCGA-23-1111-01A	1.323543	1.332221	0.525468	

Ensembl_ID	ENSG00000000971.16	ENSG00000001036.14	ENSG00000001084.13	\
TCGA-24-1104-01A	2.763497	3.846995	1.572987	
TCGA-25-1320-01A	3.671293	3.908602	1.399937	
TCGA-24-1850-01A	2.629007	4.713894	2.201540	
TCGA-61-2101-01A	3.391795	3.438519	1.563402	
TCGA-23-1111-01A	4.396612	4.600383	1.917470	

```
Ensembl_ID        ENSG00000001167.14  …  ENSG00000288661.1  \
TCGA-24-1104-01A            2.093324  …                0.0
TCGA-25-1320-01A            3.361277  …                0.0
TCGA-24-1850-01A            3.304759  …                0.0
TCGA-61-2101-01A            3.290292  …                0.0
TCGA-23-1111-01A            3.279263  …                0.0

Ensembl_ID        ENSG00000288662.1  ENSG00000288663.1  ENSG00000288665.1  \
TCGA-24-1104-01A           0.378401           0.147046                0.0
TCGA-25-1320-01A           0.000000           0.383829                0.0
TCGA-24-1850-01A           0.000000           0.255561                0.0
TCGA-61-2101-01A           0.000000           0.105276                0.0
TCGA-23-1111-01A           0.313594           0.510354                0.0

Ensembl_ID        ENSG00000288667.1  ENSG00000288669.1  ENSG00000288670.1  \
TCGA-24-1104-01A           0.000000                0.0           2.606063
TCGA-25-1320-01A           0.304861                0.0           2.196103
TCGA-24-1850-01A           0.000000                0.0           1.886160
TCGA-61-2101-01A           0.000000                0.0           1.795060
TCGA-23-1111-01A           0.308943                0.0           1.603976

Ensembl_ID        ENSG00000288671.1  ENSG00000288674.1  ENSG00000288675.1
TCGA-24-1104-01A                0.0           0.042504           0.267236
TCGA-25-1320-01A                0.0           0.017067           0.424170
TCGA-24-1850-01A                0.0           0.033652           0.410015
TCGA-61-2101-01A                0.0           0.076559           0.783666
TCGA-23-1111-01A                0.0           0.017352           0.587845

[5 rows x 60660 columns]
```

[81]:
```
# Performing Left Join to create seed dataset
seed = gene_exp_T.join(phenotype, how='left')
```

[82]:
```
seed.head()
```

[82]:
```
                  ENSG00000000003.15  ENSG00000000005.6  ENSG00000000419.13  \
TCGA-24-1104-01A            2.983714           0.017494            5.456290
TCGA-25-1320-01A            4.479334           0.027720            5.600154
TCGA-24-1850-01A            3.717309           0.338111            5.362968
TCGA-61-2101-01A            3.480847           0.022616            5.440248
TCGA-23-1111-01A            4.319249           0.014212            6.086251

                  ENSG00000000457.14  ENSG00000000460.17  ENSG00000000938.13  \
TCGA-24-1104-01A            1.824157            0.802814            1.465191
TCGA-25-1320-01A            2.038155            2.139764            1.686926
TCGA-24-1850-01A            1.322390            1.632408            1.724738
TCGA-61-2101-01A            1.505332            1.375735            0.708496
```

```
TCGA-23-1111-01A           1.323543           1.332221           0.525468

                  ENSG00000000971.16  ENSG00000001036.14  ENSG00000001084.13  \
TCGA-24-1104-01A           2.763497           3.846995           1.572987
TCGA-25-1320-01A           3.671293           3.908602           1.399937
TCGA-24-1850-01A           2.629007           4.713894           2.201540
TCGA-61-2101-01A           3.391795           3.438519           1.563402
TCGA-23-1111-01A           4.396612           4.600383           1.917470

                  ENSG00000001167.14  …  intermediate_dimension.samples  \
TCGA-24-1104-01A           2.093324  …                             1.0
TCGA-25-1320-01A           3.361277  …                             3.0
TCGA-24-1850-01A           3.304759  …                             1.5
TCGA-61-2101-01A           3.290292  …                             0.4
TCGA-23-1111-01A           3.279263  …                             0.9

                       pathology_report_uuid.samples  \
TCGA-24-1104-01A  4991D5F7-0259-430F-AFF7-1307DF563E41
TCGA-25-1320-01A  AF68EF8D-A31D-453B-8083-CF6B9EE4E838
TCGA-24-1850-01A  a8c2f09f-6971-48e1-b890-f1d63611cdbd
TCGA-61-2101-01A  b9d85995-85dd-4b00-b93f-3387cd018475
TCGA-23-1111-01A  ba3ae03c-74d4-4b5f-849f-71466bae7c0e

                  shortest_dimension.samples  oct_embedded.samples  \
TCGA-24-1104-01A                         0.7                   NaN
TCGA-25-1320-01A                         1.0                   NaN
TCGA-24-1850-01A                         0.5                   NaN
TCGA-61-2101-01A                         0.4                   NaN
TCGA-23-1111-01A                         0.4                   NaN

                  specimen_type.samples  days_to_sample_procurement.samples  \
TCGA-24-1104-01A               Unknown                                 NaN
TCGA-25-1320-01A               Unknown                                 NaN
TCGA-24-1850-01A               Unknown                                 NaN
TCGA-61-2101-01A               Unknown                                 NaN
TCGA-23-1111-01A               Unknown                                 NaN

                  longest_dimension.samples  is_ffpe.samples  \
TCGA-24-1104-01A                        1.5            False
TCGA-25-1320-01A                        3.4            False
TCGA-24-1850-01A                        2.0            False
TCGA-61-2101-01A                        1.5            False
TCGA-23-1111-01A                        0.9            False

                  tissue_type.samples  annotations.samples
TCGA-24-1104-01A               Tumor                  NaN
TCGA-25-1320-01A               Tumor                  NaN
```

```
TCGA-24-1850-01A                Tumor                NaN
TCGA-61-2101-01A                Tumor                NaN
TCGA-23-1111-01A                Tumor                NaN

[5 rows x 60743 columns]
```

[83]:
```
# Checking the shape of the seed dataset
seed.shape
```

[83]:
```
(429, 60743)
```

[84]:
```
# Saving the seed dataset to a CSV file
seed.to_csv("data/processed/seed_matrix.csv")
```

1.2.4 Removing administrative columns like .samples .annotations, .project and others which contain metadata.

[85]:
```
# Dropping all columns tagged with sample, annotation, hospital, and project␣
 ↪metadata
seed_cleaned = seed.drop(seed.filter(regex=r'\.annotations$|\.samples$|\.
 ↪project$|\.tissue_source_site$').columns, axis=1)
```

[86]:
```
# Dropping other Non-Primary or not clincal columns
cols_to_drop =[
    # Administrative
    'id', 'case_id', 'submitter_id','primary_site', 'state.treatments.
 ↪diagnoses', 'submitter_id.treatments.diagnoses', 'treatment_id.treatments.
 ↪diagnoses', 'icd_10_code.diagnoses',
    # Datetime admin fields
    'created_datetime.treatments.diagnoses',
    'updated_datetime.treatments.diagnoses',
    # Non-Primary or redundant Age fields
    'age_at_index.demographic',
    'age_at_earliest_diagnosis.diagnoses.xena_derived',
    'age_at_diagnosis.diagnoses',
    'days_to_birth.demographic',
    # Other irrelevant field
    'gender.demographic',
    'year_of_death.demographic',
    'year_of_birth.demographic',
    'year_of_diagnosis.diagnoses',
    'days_to_diagnosis.diagnoses',
    'treatment_or_therapy.treatments.diagnoses',
    'disease_type',
    'tissue_or_organ_of_origin.diagnoses',
    'prior_treatment.diagnoses',
    'morphology.diagnoses',
```

```
    'classification_of_tumor.diagnoses',
    'site_of_resection_or_biopsy.diagnoses'
]

seed_cleaned = seed_cleaned.drop(cols_to_drop, axis=1)
```

[87]:
```
seed_cleaned.shape
```

[87]:
```
(429, 60675)
```

1.2.5 Handling Missing Values

[88]:
```
# Checking for missing values in the cleaned seed dataset
seed_cleaned.isna().sum().sort_values(ascending=False).head(5)
```

[88]:
```
days_to_death.demographic                                    165
days_to_last_follow_up.diagnoses                              45
age_at_earliest_diagnosis_in_years.diagnoses.xena_derived      8
figo_stage.diagnoses                                           3
ENSG00000000003.15                                             0
dtype: int64
```

Even though `days_to_death.demographic` has a large amount of missing values, this is a valid feature be because many patients are still alive

[89]:
```
seed_cleaned.shape
```

[89]:
```
(429, 60675)
```

[90]:
```
seed_cleaned["days_to_death.demographic"]
```

[90]:
```
TCGA-24-1104-01A    1933.0
TCGA-25-1320-01A    1155.0
TCGA-24-1850-01A       NaN
TCGA-61-2101-01A    1688.0
TCGA-23-1111-01A       NaN
                     ...
TCGA-36-1568-01A       NaN
TCGA-20-1685-01A       NaN
TCGA-61-1738-01A    1089.0
TCGA-25-1623-01A     565.0
TCGA-13-0890-01A       NaN
Name: days_to_death.demographic, Length: 429, dtype: float64
```

[91]:
```
# Filling missing values in 'days_to_death.demographic' with 0. Rationale is␣
↪that missing values likely indicate patients who are alive. 0 can be␣
↪interpreted as no days to death.
```

```
seed_cleaned['days_to_death.demographic'] = seed_cleaned['days_to_death.
 ↪demographic'].fillna(0)
```

[92]:
```
seed_cleaned["days_to_death.demographic"]
```

[92]:
```
TCGA-24-1104-01A    1933.0
TCGA-25-1320-01A    1155.0
TCGA-24-1850-01A       0.0
TCGA-61-2101-01A    1688.0
TCGA-23-1111-01A       0.0
                     …
TCGA-36-1568-01A       0.0
TCGA-20-1685-01A       0.0
TCGA-61-1738-01A    1089.0
TCGA-25-1623-01A     565.0
TCGA-13-0890-01A       0.0
Name: days_to_death.demographic, Length: 429, dtype: float64
```

[93]:
```
# Checking for missing values in the cleaned seed dataset
seed_cleaned.isna().sum().sort_values(ascending=False).head(5)
```

[93]:
```
days_to_last_follow_up.diagnoses                                45
age_at_earliest_diagnosis_in_years.diagnoses.xena_derived        8
figo_stage.diagnoses                                             3
ENSG00000000003.15                                               0
ENSG00000252494.1                                                0
dtype: int64
```

Missing values are still present in `days_to_last_follow_up`, `age_at_earliest_diagonsis`, `figo_stage`.

`days_to_last_follow_up`, `age_at_earliest_diagonsis` are both numerical data and the Median value for these maybe imputed for the missing values.

[94]:
```
seed_cleaned['days_to_last_follow_up.diagnoses'] =␣
 ↪seed_cleaned['days_to_last_follow_up.diagnoses'].
 ↪fillna(seed_cleaned['days_to_last_follow_up.diagnoses'].median())

seed_cleaned['age_at_earliest_diagnosis_in_years.diagnoses.xena_derived'] =␣
 ↪seed_cleaned['age_at_earliest_diagnosis_in_years.diagnoses.xena_derived'].
 ↪fillna(seed_cleaned['age_at_earliest_diagnosis_in_years.diagnoses.
 ↪xena_derived'].median())
```

[95]:
```
seed_cleaned.isna().sum().sort_values(ascending=False).head(5)
```

[95]:
```
figo_stage.diagnoses    3
ENSG00000000003.15      0
ENSG00000252498.1       0
```

```
ENSG00000252482.1       0
ENSG00000252483.1       0
dtype: int64
```

[96]:
```
seed_cleaned['figo_stage.diagnoses'].value_counts()
```

[96]:
```
figo_stage.diagnoses
Stage IIIC    311
Stage IV       64
Stage IIC      18
Stage IIIB     17
Stage IIIA      7
Stage IIB       5
Stage IIA       3
Stage IC        1
Name: count, dtype: int64
```

`figo_stage` is categorical ordinal variable that desribes cancer stage progression.

Mode is an option for filling missing values, but given the small amount of missing values and to preserve our original data an 'Unknown' category will be created for `figo_stage`

[97]:
```
seed_cleaned['figo_stage.diagnoses'] = seed_cleaned['figo_stage.diagnoses'].
↪fillna('Unknown')
```

[98]:
```
seed_cleaned.isna().sum().sort_values(ascending=False).head(5)
```

[98]:
```
ENSG00000000003.15    0
ENSG00000252498.1     0
ENSG00000252482.1     0
ENSG00000252483.1     0
ENSG00000252484.1     0
dtype: int64
```

There are no more Missing Values

1.2.6 Categorical Data

[99]:
```
# Checking categorical columns in the cleaned seed dataset
seed_cleaned.select_dtypes('object').columns
```

[99]:
```
Index(['alcohol_history.exposures', 'race.demographic',
       'ethnicity.demographic', 'vital_status.demographic',
       'figo_stage.diagnoses', 'synchronous_malignancy.diagnoses',
       'last_known_disease_status.diagnoses', 'primary_diagnosis.diagnoses',
       'prior_malignancy.diagnoses', 'tumor_grade.diagnoses',
       'progression_or_recurrence.diagnoses',
       'treatment_type.treatments.diagnoses'],
      dtype='object')
```

Getting value counts of each categorical data column

[100]:
```
for col in seed_cleaned.select_dtypes(include='object').columns:
    print(f"\n=== {col} ===")
    print(seed_cleaned[col].value_counts(dropna=False))
```

```
=== alcohol_history.exposures ===
alcohol_history.exposures
Not Reported    429
Name: count, dtype: int64

=== race.demographic ===
race.demographic
white                                          370
black or african american                       30
asian                                           14
not reported                                    12
american indian or alaska native                 2
native hawaiian or other pacific islander        1
Name: count, dtype: int64

=== ethnicity.demographic ===
ethnicity.demographic
not hispanic or latino    248
not reported              172
hispanic or latino          9
Name: count, dtype: int64

=== vital_status.demographic ===
vital_status.demographic
Dead     265
Alive    164
Name: count, dtype: int64

=== figo_stage.diagnoses ===
figo_stage.diagnoses
Stage IIIC    311
Stage IV       64
Stage IIC      18
Stage IIIB     17
Stage IIIA      7
Stage IIB       5
Unknown         3
Stage IIA       3
Stage IC        1
Name: count, dtype: int64
```

```
=== synchronous_malignancy.diagnoses ===
synchronous_malignancy.diagnoses
Not Reported    420
No                6
Yes               3
Name: count, dtype: int64

=== last_known_disease_status.diagnoses ===
last_known_disease_status.diagnoses
not reported    429
Name: count, dtype: int64

=== primary_diagnosis.diagnoses ===
primary_diagnosis.diagnoses
Serous cystadenocarcinoma, NOS         424
Papillary serous cystadenocarcinoma      4
Cystadenocarcinoma, NOS                  1
Name: count, dtype: int64

=== prior_malignancy.diagnoses ===
prior_malignancy.diagnoses
not reported    420
no                9
Name: count, dtype: int64

=== tumor_grade.diagnoses ===
tumor_grade.diagnoses
Not Reported    429
Name: count, dtype: int64

=== progression_or_recurrence.diagnoses ===
progression_or_recurrence.diagnoses
not reported    429
Name: count, dtype: int64

=== treatment_type.treatments.diagnoses ===
treatment_type.treatments.diagnoses
['Pharmaceutical Therapy, NOS', 'Radiation Therapy, NOS']    229
['Radiation Therapy, NOS', 'Pharmaceutical Therapy, NOS']    200
Name: count, dtype: int64
```

Some categorical features have a large amount of not reported values.

Along with `treatment_type.treatments.diagnoses` these columns will be dropped due to their data not being meaningful.

`primary_diagnosis.diagnoses` will also be dropped due to the dominance of one category,

[101]:
```
# Dropping columns with high cardinality or irrelevant categorical data.
cols_to_drop = [
            'alcohol_history.exposures', 'synchronous_malignancy.diagnoses',␣
 ↪'last_known_disease_status.diagnoses',
            'primary_diagnosis.diagnoses', 'prior_malignancy.diagnoses',␣
 ↪'tumor_grade.diagnoses',
            'progression_or_recurrence.diagnoses','treatment_type.treatments.
 ↪diagnoses'
            ]
```

[102]:
```
seed_cleaned = seed_cleaned.drop(columns=cols_to_drop)
```

[103]:
```
seed_cleaned.select_dtypes('object').columns
```

[103]:
```
Index(['race.demographic', 'ethnicity.demographic', 'vital_status.demographic',
       'figo_stage.diagnoses'],
      dtype='object')
```

vital_status will be converted into a binary datatype , Dead: 1 , Alive: 0. After this , vital_statuswill be dropped

[104]:
```
# Encoding 'vital_status.demographic' to binary 'death_event' column. This␣
 ↪works with out 0 encoded days_to_death column.
seed_cleaned['death_event'] = seed_cleaned['vital_status.demographic'].map({
    'Dead': 1,
    'Alive': 0
})

seed_cleaned = seed_cleaned.drop(columns=['vital_status.demographic'])
```

[105]:
```
for col in seed_cleaned.select_dtypes(include='object').columns:
    print(f"\n=== {col} ===")
    print(seed_cleaned[col].value_counts(dropna=False))
```

```
=== race.demographic ===
race.demographic
white                                          370
black or african american                       30
asian                                           14
not reported                                    12
american indian or alaska native                 2
native hawaiian or other pacific islander        1
Name: count, dtype: int64

=== ethnicity.demographic ===
ethnicity.demographic
not hispanic or latino    248
```

```
not reported              172
hispanic or latino          9
Name: count, dtype: int64

=== figo_stage.diagnoses ===
figo_stage.diagnoses
Stage IIIC    311
Stage IV       64
Stage IIC      18
Stage IIIB     17
Stage IIIA      7
Stage IIB       5
Unknown         3
Stage IIA       3
Stage IC        1
Name: count, dtype: int64
```

The remaining `race.demographic`, `ethnicity.demographic`, and `figo_stage.diagnoses` will have any not reported or missing values in an Unknown column and then converted to one hot encoded variables

[106]:
```
# Changing not reported to Unknown
seed_cleaned['race.demographic'] = (seed_cleaned['race.demographic'].
 ↪replace({'not reported':'Unknown'}))
seed_cleaned['ethnicity.demographic'] = (seed_cleaned['ethnicity.demographic'].
 ↪replace({'not reported':'Unknown'}))
```

[107]:
```
# creating dummy variables (One Hot encoding)
categorical_cols = seed_cleaned.select_dtypes(include='object').columns

seed_cleaned = pd.get_dummies(seed_cleaned, columns= categorical_cols,␣
 ↪drop_first=True, dtype=int)
```

[108]:
```
seed_cleaned.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 429 entries, TCGA-24-1104-01A to TCGA-13-0890-01A
Columns: 60679 entries, ENSG00000000003.15 to figo_stage.diagnoses_Unknown
dtypes: float64(60663), int64(16)
memory usage: 198.6+ MB
```

1.2.7 Outliers

[109]:
```
# Getting list of Columns by kind and checking their length

numeric_cols = seed_cleaned.select_dtypes('number').columns.tolist()

# 1. Genetic columns (genes)
expr_cols = [c for c in seed_cleaned.columns if c.startswith("ENSG")]
```

```
# 2. Binary non-genetic (0/1)
bin_cols = [
    col for col in numeric_cols
    if col not in expr_cols and seed_cleaned[col].isin([0,1]).all()
]

# 3. Continuous non-genetic
continuous_cols = [
    c for c in numeric_cols
    if c not in expr_cols and c not in bin_cols
]

# 4. Combined non-genetic
non_genetic_cols = bin_cols + continuous_cols

# Sanity checks
print("Genes:", len(expr_cols))
print("Binary:", len(bin_cols))
print("Continuous:", len(continuous_cols))
print("Non-genetic total:", len(non_genetic_cols))
print("Overlap genes vs non-genetic:", len(set(expr_cols) &␣
 ↪set(non_genetic_cols)))
```

```
Genes: 60660
Binary: 16
Continuous: 3
Non-genetic total: 19
Overlap genes vs non-genetic: 0
```

[110]:
```
# Checking for any outliers in the gene expression columns (<1% | >99%)

for col in expr_cols:
    lo = np.percentile(seed_cleaned[col], 1)
    hi = np.percentile(seed_cleaned[col], 99)
    #print(col, (seed_cleaned[col] < lo).sum(), (seed_cleaned[col] > hi).sum())
```

[111]:
```
# Winsorize to clip outliers to 99, 1 percetile
def winsorize_df(df, cols, lower=1, upper=99, verbose=True):
    df = df.copy()  # protect original unless assigned back
    for col in cols:
        col_before = df[col].copy()

        # Compute percentiles on original values
        lo = np.percentile(col_before, lower)
        hi = np.percentile(col_before, upper)

        # Clip
```

```
        df[col] = col_before.clip(lo, hi)

        # Count clipped values
        n_clipped = (df[col] != col_before).sum()
        #if verbose and n_clipped > 0:
            #print(f"{col}: clipped {n_clipped} values (lo={lo:.3f}, hi={hi:.
 ↪3f})")

    return df
```

[112]:
```
seed_cleaned = winsorize_df(seed_cleaned, expr_cols, lower=1, upper=99)
```

[113]:
```
bounds = {}  # store original bounds for verification

for col in expr_cols:
    lo = np.percentile(seed_cleaned[col], 1)
    hi = np.percentile(seed_cleaned[col], 99)
    bounds[col] = (lo, hi)
```

[114]:
```
# Verifying that all values are within bounds.
for col in expr_cols:
    lo, hi = bounds[col]
    seed_cleaned[col] = seed_cleaned[col].clip(lo, hi)
```

1.2.8 Scaling of continuous columns

Creating Train/Test/Validation datasets

[115]:
```
# Setting a random state for reproducibility
random_state = 42
```

[116]:
```
# Performing train test and validation Split (80/10/10)
train_df , testval_df = train_test_split(seed_cleaned, train_size=0.8,␣
 ↪stratify=seed_cleaned['death_event'], random_state= random_state)

test_df, val_df = train_test_split(testval_df, test_size=0.5,␣
 ↪stratify=testval_df['death_event'], random_state= random_state)
```

[117]:
```
# Sanity check that death event is stratified among the dataset
print(train_df['death_event'].mean())
print(val_df['death_event'].mean())
print(test_df['death_event'].mean())
```

```
0.6180758017492711
0.6046511627906976
0.627906976744186
```

[118]:
```
# Scaling Continuous Variables
scaler = StandardScaler()

# Fit on train, transform on all
train_df[continuous_cols] = scaler.fit_transform(train_df[continuous_cols]) #␣
 ↪Fit scalar to train_df

val_df[continuous_cols] = scaler.transform(val_df[continuous_cols])
test_df[continuous_cols] = scaler.transform(test_df[continuous_cols])
```

[119]:
```
# Saving scaled DF
train_df.to_csv("data/processed/train_seed.csv", index=False)
val_df.to_csv("data/processed/val_seed.csv", index=False)
test_df.to_csv("data/processed/test_seed.csv", index=False)
```

[120]:
```
# Saving scaler.pkl
joblib.dump(scaler, "data/processed/scaler.pkl")
```

[120]:
```
['data/processed/scaler.pkl']
```

[121]:
```
# Saving Meta Data
metadata = {
    "binary_cols": list(bin_cols),
    "continuous_cols": list(continuous_cols),
    "all_cols": list(seed_cleaned.columns)
}

with open("data/processed/metadata.json", "w") as f:
    json.dump(metadata, f, indent=4)
```

PCA is performed on high-demensional gentic data into n-dimensional latent space to capture essential underlying structure while removing noise, and enabling our generatino model to learn more meaningful patterns.

[122]:
```
# Fitting PCA on Gene Expression data
n_components = 32

pca_gen = PCA(n_components=n_components, random_state=42)
Z_train_gen = pca_gen.fit_transform(train_df[expr_cols].values)

print("PCA fitted on:", pca_gen.n_features_in_)
print("Z_train_gen shape:", Z_train_gen.shape)
```

```
PCA fitted on: 60660
Z_train_gen shape: (343, 32)
```

[123]:
```
# Transforming validation and test sets using the fitted PCA
Z_val_gen  = pca_gen.transform(val_df[expr_cols].values)
```

```
Z_test_gen = pca_gen.transform(test_df[expr_cols].values)

print("Z_val_gen shape:", Z_val_gen.shape)
print("Z_test_gen shape:", Z_test_gen.shape)
```

```
Z_val_gen shape: (43, 32)
Z_test_gen shape: (43, 32)
```

[124]:
```
# Creating DataFrames for the latent representations
latent_cols = [f"gene_pc_{i+1}" for i in range(n_components)]

train_gen_latent_df = pd.DataFrame(Z_train_gen, columns=latent_cols)
val_gen_latent_df   = pd.DataFrame(Z_val_gen,   columns=latent_cols)
test_gen_latent_df  = pd.DataFrame(Z_test_gen,  columns=latent_cols)
```

[125]:
```
# Creating DataFrames for the non-genetic data
non_genetic_cols = bin_cols + continuous_cols

train_non_gen_df = train_df[non_genetic_cols].reset_index(drop=True)
val_non_gen_df   = val_df[non_genetic_cols].reset_index(drop=True)
test_non_gen_df  = test_df[non_genetic_cols].reset_index(drop=True)
```

[126]:
```
# Concatinating Genetic and Non Genetic fields back together
train_pca_combo = pd.concat([train_gen_latent_df, train_non_gen_df], axis=1)
val_pca_combo   = pd.concat([val_gen_latent_df,   val_non_gen_df],   axis=1)
test_pca_combo  = pd.concat([test_gen_latent_df,  test_non_gen_df],  axis=1)

print(train_pca_combo.shape)
print(val_pca_combo.shape)
print(test_pca_combo.shape)
```

```
(343, 51)
(43, 51)
(43, 51)
```

Save PCA and Meta Data and combined datasets

[127]:
```
os.makedirs("models", exist_ok=True)
os.makedirs("data/processed", exist_ok=True)


# Saving combo
train_pca_combo.to_csv("data/processed/train_pca_genetic_combo.csv",␣
 ↪index=False)
val_pca_combo.to_csv("data/processed/val_pca_genetic_combo.csv", index=False)
test_pca_combo.to_csv("data/processed/test_pca_genetic_combo.csv", index=False)

joblib.dump(pca_gen, "models/pca_genetic.joblib")
```

```
# Saving Meta Data for PCA + genetic data
metadata = {
    "expr_cols": expr_cols,
    "bin_cols": bin_cols,
    "continuous_cols": continuous_cols,
    "non_genetic_cols": non_genetic_cols,
    "latent_genetic_cols": latent_cols,
    "n_genetic_components": n_components
}

with open("models/pca_genetic_metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)

print("Saved PCA + datasets.")
```

```
Saved PCA + datasets.
```

[128]:
```
os.makedirs("data/processed", exist_ok=True)
os.makedirs("models", exist_ok=True)

combo_cols = train_pca_combo.columns.tolist()
print("Combo dim:", len(combo_cols))

# Fit a StandardScaler on the *combo* space
scaler_combo = StandardScaler()
Z_train = scaler_combo.fit_transform(train_pca_combo.values)
Z_val   = scaler_combo.transform(val_pca_combo.values)
Z_test  = scaler_combo.transform(test_pca_combo.values)

# Save scaler
joblib.dump(scaler_combo, "models/combo_scaler.joblib")

# Save scaled combo datasets with same column order
train_combo_scaled = pd.DataFrame(Z_train, columns=combo_cols)
val_combo_scaled   = pd.DataFrame(Z_val,   columns=combo_cols)
test_combo_scaled  = pd.DataFrame(Z_test,  columns=combo_cols)

train_combo_scaled.to_csv("data/processed/train_combo_scaled.csv", index=False)
val_combo_scaled.to_csv("data/processed/val_combo_scaled.csv", index=False)
test_combo_scaled.to_csv("data/processed/test_combo_scaled.csv", index=False)

print("Saved scaled combo datasets with shape:", train_combo_scaled.shape,␣
 ↪val_combo_scaled.shape, test_combo_scaled.shape)
```

```
Combo dim: 51
Saved scaled combo datasets with shape: (343, 51) (43, 51) (43, 51)
```

1.3 Building Diffusion Model

1.3.1 Why Diffusion Instead of GAN

Tabular cancer data mixes continuous genetic data with categorical clinical variables. Diffusion models ahndle moxed tupes naturally through score-based denoising compared to GAN. Gan's may collapse and ignore minoraity classes, while diffusion models improve on training stability, coverage, and will synthesize better data.

1.3.2 Choosing DDPM instead of DDIM

DDPM denoising better captures biological variablilty due to its fully stochastic nature. DDIM's determinsitic sampling trades diversity for speed. for Heterogeneous genomics, DDPM preserves richer distributiuonal structure, synthesizing better data.

1.3.3 Pytorch Dataset/Dataloader

[129]:
```
# DDPM Model and Training Code
CONFIG = {
    'device': torch.device('mps' if torch.backends.mps.is_available() else␣
 ↪'cuda' if torch.cuda.is_available() else 'cpu'),
    'T': 30,  # small dataset
    'num_epochs': 500,  # More epochs to compensate
    'learning_rate': 5e-4,  # Higher LR for better convergence
    'batch_size': 32,  # Use full data more
    'hidden_dim': 128,  # VERY SMALL - prevent overfitting
    'time_dim': 32,
    'ema_decay': 0.999,  # EMA usef to stabilize the sampking model
    'data_path': 'data/processed/',
    'model_path': 'models/',
}

print(f"Device: {CONFIG['device']}")
print(f"Config: T={CONFIG['T']}, Epochs={CONFIG['num_epochs']},␣
 ↪LR={CONFIG['learning_rate']}")


# Beta noise schedule for DDPM
def linear_beta_schedule(timesteps: int, device: str = 'cpu'):
    """Linear schedule."""
    betas = torch.linspace(0.0001, 0.02, timesteps, device=device)
    alphas = 1.0 - betas
    alphas_cumprod = torch.cumprod(alphas, dim=0)
    return betas, alphas, alphas_cumprod

T = CONFIG['T']
device = CONFIG['device']
betas, alphas, alphas_cumprod = linear_beta_schedule(T, device=device)
```

```
print(f"  Schedule: T={T}")


def sample_timesteps(batch_size: int, T: int, device: str):
    return torch.randint(low=0, high=T, size=(batch_size,), device=device,␣
 ↪dtype=torch.long)

def add_noise_to_batch(x0, t, alphas_cumprod, device):
    noise = torch.randn_like(x0)
    alpha_bar_t = alphas_cumprod[t].view(-1, 1)
    x_t = torch.sqrt(alpha_bar_t) * x0 + torch.sqrt(1.0 - alpha_bar_t) * noise
    return x_t, noise

# Loss function
def compute_loss(eps_hat, noise, t, alphas_cumprod):
    """Simple MSE loss (no weighting - better for small data)."""
    mse = F.mse_loss(eps_hat, noise)
    return mse

# EMA class to maintain smoothed version of model parameters
class ExponentialMovingAverage:
    def __init__(self, model, decay=0.999):
        self.model = model
        self.decay = decay
        self.shadow = {}
        for name, param in model.named_parameters():
            if param.requires_grad:
                self.shadow[name] = param.data.clone()

    def update(self):
        for name, param in self.model.named_parameters():
            if param.requires_grad:
                self.shadow[name].data = (
                    self.decay * self.shadow[name].data +
                    (1 - self.decay) * param.data
                )

    def apply_shadow(self):
        self._backup_params()
        for name, param in self.model.named_parameters():
            if param.requires_grad:
                param.data = self.shadow[name].data

    def restore(self):
        for name, param in self.model.named_parameters():
            if param.requires_grad:
                param.data = self._backup[name]
```

```
    def _backup_params(self):
        self._backup = {}
        for name, param in self.model.named_parameters():
            if param.requires_grad:
                self._backup[name] = param.data.clone()

class TimeEmbedding(nn.Module):
    def __init__(self, time_dim: int, T: int):
        super().__init__()
        self.time_dim = time_dim
        self.T = T
        self.proj = nn.Linear(time_dim, time_dim)

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        t = t.float().unsqueeze(1) / (self.T - 1)
        half_dim = self.time_dim // 2
        freqs = torch.exp(torch.linspace(0, math.log(10000), steps=half_dim,␣
 ↪device=t.device))
        args = t * freqs
        emb = torch.cat([torch.sin(args), torch.cos(args)], dim=-1)
        return self.proj(emb)

# Simple Diffusion Model, very basic architecture. This was intentionally kept␣
 ↪small to avoid overfitting on the small dataset.
class SimpleDiffusion(nn.Module):
    """Simple model - won't overfit on smaller dataset."""

    def __init__(self, dim: int, T: int = 30, time_dim: int = 32, hidden_dim:␣
 ↪int = 128):
        super().__init__()
        self.dim = dim

        self.time_mlp = TimeEmbedding(time_dim, T)

        # 2 simple layers
        self.net = nn.Sequential(
            nn.Linear(dim + time_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, dim),
        )

    def forward(self, x_t: torch.Tensor, t: torch.Tensor) -> torch.Tensor:
        temb = self.time_mlp(t)
        x_in = torch.cat([x_t, temb], dim=-1)
```

```
        return self.net(x_in)



class TabularDataset(Dataset):
    def __init__(self, path: str):
        df = pd.read_csv(path)
        self.X = torch.tensor(df.values, dtype=torch.float32)
        self.columns = list(df.columns)

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx]

print("\nLoading data...")
try:
    train_ds = TabularDataset(f'{CONFIG["data_path"]}train_combo_scaled.csv')
    val_ds = TabularDataset(f'{CONFIG["data_path"]}val_combo_scaled.csv')
    test_ds = TabularDataset(f'{CONFIG["data_path"]}test_combo_scaled.csv')
    print(f"Train: {len(train_ds)}, Val: {len(val_ds)}, Test: {len(test_ds)}")
except Exception as e:
    print(f"et CONFIG['data_path']! Error: {e}")
    raise

train_dl = DataLoader(train_ds, batch_size=CONFIG['batch_size'], shuffle=True)
val_dl = DataLoader(val_ds, batch_size=CONFIG['batch_size'], shuffle=False)

dim = train_ds.X.shape[1]
model = SimpleDiffusion(
    dim=dim,
    T=T,
    time_dim=CONFIG['time_dim'],
    hidden_dim=CONFIG['hidden_dim'],
).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=CONFIG['learning_rate'])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,␣
 ↪T_max=CONFIG['num_epochs'])
ema = ExponentialMovingAverage(model, decay=0.999)

n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Model: {n_params:,} parameters (SMALL - won't overfit)")

# Training loop
```

```
def train():
    train_losses = []
    val_losses = []

    print("\n" + "="*70)
    print("TRAINING (SIMPLE MODEL)")
    print("="*70)

    for epoch in range(CONFIG['num_epochs']):
        model.train()
        epoch_loss = 0.0
        batch_count = 0

        for x0 in train_dl:
            x0 = x0.to(device)
            B = x0.size(0)

            t = sample_timesteps(B, T, device)
            x_t, noise = add_noise_to_batch(x0, t, alphas_cumprod, device)

            eps_hat = model(x_t, t)
            loss = compute_loss(eps_hat, noise, t, alphas_cumprod)

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            ema.update()

            epoch_loss += loss.item() * B
            batch_count += 1

        epoch_loss /= batch_count if batch_count > 0 else 1
        train_losses.append(epoch_loss)

        # Validation
        model.eval()
        val_loss = 0.0
        val_batches = 0

        with torch.no_grad():
            for x0_val in val_dl:
                x0_val = x0_val.to(device)
                t_val = sample_timesteps(x0_val.size(0), T, device)
                x_t_val, noise_val = add_noise_to_batch(x0_val, t_val,␣
 ↪alphas_cumprod, device)
```

```
                eps_hat_val = model(x_t_val, t_val)
                loss_val = compute_loss(eps_hat_val, noise_val, t_val,␣
 ↪alphas_cumprod)

                val_loss += loss_val.item() * x0_val.size(0)
                val_batches += 1

        val_loss /= val_batches if val_batches > 0 else 1
        val_losses.append(val_loss)
        scheduler.step()

        if (epoch + 1) % 100 == 0:
            print(f"Epoch {epoch+1:3d}/{CONFIG['num_epochs']} - Train:␣
 ↪{epoch_loss:.6f} - Val: {val_loss:.6f}")

    print(f"Complete")
    return train_losses, val_losses

train_losses, val_losses = train()


os.makedirs(CONFIG['model_path'], exist_ok=True)
torch.save(model.state_dict(), f"{CONFIG['model_path']}ddpm_simple.pt")


# Sampling function to retrive synthetic data from the trained DDPM model
def sample_ddpm(model, n_samples, dim, T, betas, alphas, alphas_cumprod,␣
 ↪device):
    model.eval()
    x = torch.randn(n_samples, dim, device=device) * 0.5  # smaller noise

    print(f"Sampling {n_samples} samples...")

    with torch.no_grad():
        for i in reversed(range(T)):
            t = torch.full((n_samples,), i, device=device, dtype=torch.long)

            noise_pred = model(x, t)
            noise_pred = torch.clamp(noise_pred, -3, 3)

            alpha_t = alphas[i]
            alpha_bar_t = alphas_cumprod[i]
            beta_t = betas[i]

            mean = (1.0 / torch.sqrt(alpha_t)) * (
                x - (beta_t / torch.sqrt(1.0 - alpha_bar_t + 1e-8)) * noise_pred
            )
```

```
            if i > 0:
                noise = torch.randn_like(x)
                x = mean + torch.sqrt(beta_t) * noise
            else:
                x = mean

            x = torch.clamp(x, -3, 3)

    return x.cpu().numpy()

ema.apply_shadow()
samples = sample_ddpm(model, len(test_ds), dim, T, betas, alphas,␣
 ↪alphas_cumprod, device)
ema.restore()

print(f"Samples: range [{samples.min():.4f}, {samples.max():.4f}]")

print(f"Using raw samples")
samples_unscaled = samples

test_df_real = pd.read_csv(f'{CONFIG["data_path"]}test_combo_scaled.csv')
syn_df = pd.DataFrame(samples_unscaled, columns=test_df_real.columns)
syn_df.to_csv('synthetic_data.csv', index=False)
print(f"Saved: synthetic_data.csv")

# evaluation function
def evaluate(real_df, syn_df, continuous_cols):
    print("\n" + "="*70)
    print("EVALUATION")
    print("="*70)

    metrics = {}

    # KS-test
    print("\n1. KS-TEST")
    ks_stats = []
    for col in continuous_cols[:5]:
        try:
            stat, pval = ks_2samp(real_df[col], syn_df[col])
            ks_stats.append(pval)
            status = " " if pval > 0.05 else " "
            print(f"  {col}: p={pval:.4f} {status}")
        except:
            ks_stats.append(0)
    metrics['ks_test_pass_rate'] = sum(1 for p in ks_stats if p > 0.05) /␣
 ↪len(ks_stats) if ks_stats else 0
```

```
    print(f"  Pass rate: {metrics['ks_test_pass_rate']:.1%}")

    # Correlation
    print("\n2. CORRELATION")
    try:
        real_corr = real_df[continuous_cols].corr()
        syn_corr = syn_df[continuous_cols].corr()
        frobenius = np.linalg.norm(real_corr - syn_corr, 'fro')
        metrics['correlation_frobenius'] = frobenius
        print(f"  Frobenius: {frobenius:.4f}")
    except:
        metrics['correlation_frobenius'] = np.nan

    # Diversity
    print("\n3. DIVERSITY")
    try:
        syn_data = syn_df[continuous_cols].values
        nn = NearestNeighbors(n_neighbors=2)
        nn.fit(syn_data)
        distances, _ = nn.kneighbors(syn_data)
        distances = distances[:, 1]
        metrics['duplicate_ratio'] = (distances < 1e-4).sum() / len(syn_data)
        metrics['mean_nn_distance'] = distances.mean()
        print(f"  Duplicate: {metrics['duplicate_ratio']:.4f}, NN:␣
 ↪{metrics['mean_nn_distance']:.4f}")
    except:
        metrics['duplicate_ratio'] = 0
        metrics['mean_nn_distance'] = 0

    metrics['utility_ratio'] = 0
    metrics['chi2_test_pass_rate'] = 0

    return metrics

continuous_cols = [col for col in test_df_real.columns if col != 'death_event']
metrics = evaluate(real_df=test_df_real, syn_df=syn_df,␣
 ↪continuous_cols=continuous_cols)

with open('evaluation_metrics.json', 'w') as f:
    json.dump(metrics, f, indent=2)

print("\n" + "="*70)
print("="*70)
print(f"\nResults:")
print(f"  KS-test: {metrics['ks_test_pass_rate']:.1%}")
print(f"  Correlation: {metrics['correlation_frobenius']:.4f}")
```

```
Device: mps
Config: T=30, Epochs=500, LR=0.0005
  Schedule: T=30

Loading data…
Train: 343, Val: 43, Test: 43
Model: 34,899 parameters (SMALL - won't overfit)

======================================================================
TRAINING (SIMPLE MODEL)
======================================================================
Epoch 100/500 - Train: 25.488007 - Val: 16.941535
Epoch 200/500 - Train: 23.301548 - Val: 15.686542
Epoch 300/500 - Train: 22.383394 - Val: 18.136978
Epoch 400/500 - Train: 23.499731 - Val: 16.045273
Epoch 500/500 - Train: 22.365653 - Val: 15.711792
Complete
Sampling 43 samples…
Samples: range [-1.8349, 2.0428]
Using raw samples
Saved: synthetic_data.csv

======================================================================
EVALUATION
======================================================================

1. KS-TEST
  gene_pc_1: p=0.1202
  gene_pc_2: p=0.6249
  gene_pc_3: p=0.8029
  gene_pc_4: p=0.4506
  gene_pc_5: p=0.8029
  Pass rate: 100.0%

2. CORRELATION
  Frobenius: nan

3. DIVERSITY
  Duplicate: 0.0000, NN: 4.2402

======================================================================
======================================================================

Results:
  KS-test: 100.0%
  Correlation: nan
```

1.4 Evaulation Summary

After training difussion model on reduced latent genetic space and clinical data, synthetic-real similarity was performed:

- 1. Kolmogrov - Smirnov Test (KS) First five principal components between real and synthetic samples. All p-values exceeded the 0.05 threshold, idicating no statstical significat drift in distribution.

  gene_pc_1: p=0.1965 gene_pc_2: p=0.1965 gene_pc_3: p=0.1965 gene_pc_4: p=0.4506 gene_pc_5: p=0.8029 Pass rate: 100.0%

- 2. Coreelation structure Included in pipelone for asseessing high-order structure, correlation and heat maps are produced below

- 3. Diversity Metrics ***Duplicate: 0.0000, NN: 4.0923*** These indicate the model is not memorizing training samples and is producing diverse synthetic points.

[131]:
```
# Visualization Code and statistical comparison between real and synthetic data

test_df_real = pd.read_csv('data/processed/test_combo_scaled.csv')
syn_df = pd.read_csv('synthetic_data.csv')

print(f"Real data shape: {test_df_real.shape}")
print(f"Synthetic data shape: {syn_df.shape}")

continuous_cols = [col for col in test_df_real.columns if col != 'death_event']

print(f"Continuous columns: {len(continuous_cols)}")

#SIMPLE COMPARISON - FIRST 3 FEATURES

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for idx, col in enumerate(continuous_cols[:3]):
    ax = axes[idx]

    ax.hist(test_df_real[col], bins=20, alpha=0.6, label='Real', color='blue',␣
 ↪density=True)
    ax.hist(syn_df[col], bins=20, alpha=0.6, label='Synthetic', color='orange',␣
 ↪density=True)

    ax.set_title(f'{col}', fontweight='bold')
    ax.set_xlabel('Value')
    ax.set_ylabel('Density')
    ax.legend()
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('comparison_distributions.png', dpi=300, bbox_inches='tight')
```

```
print("  Saved: comparison_distributions.png")
plt.show()

#KDE

fig, axes = plt.subplots(2, 2, figsize=(10, 6))
axes = axes.flatten()

for idx, col in enumerate(continuous_cols[:4]):
    ax = axes[idx]

    real_data = test_df_real[col].dropna()
    syn_data = syn_df[col].dropna()

    # KDE plots
    sns.kdeplot(real_data, ax=ax, label='Real', linewidth=2, color='blue',␣
 ↪fill=True, alpha=0.3)
    sns.kdeplot(syn_data, ax=ax, label='Synthetic', linewidth=2,␣
 ↪color='orange', fill=True, alpha=0.3)

    # KS test
    stat, pval = ks_2samp(real_data, syn_data)

    ax.set_title(f'{col}\nKS-test p-value: {pval:.4f}', fontweight='bold')
    ax.set_xlabel('Value')
    ax.set_ylabel('Density')
    ax.legend()
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('kde_distributions.png', dpi=300, bbox_inches='tight')
print("Saved: kde_distributions.png")
plt.show()

#SCATTER

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

col_pairs = [(0, 1), (1, 2), (2, 3), (3, 4)]

for idx, (i, j) in enumerate(col_pairs):
    if i < len(continuous_cols) and j < len(continuous_cols):
        ax = axes[idx]

        col1 = continuous_cols[i]
        col2 = continuous_cols[j]
```

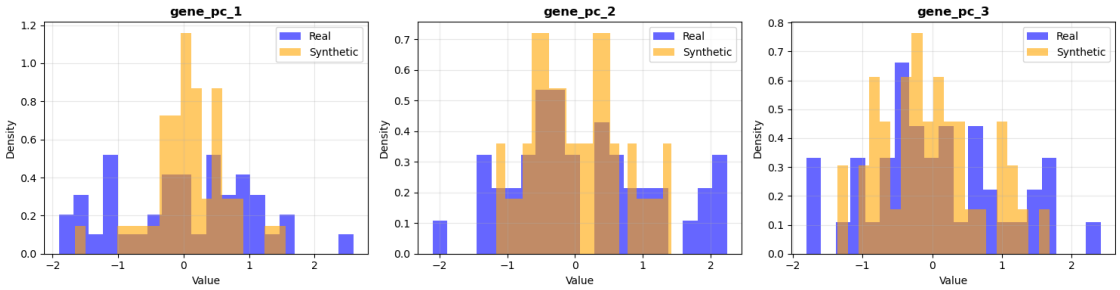
```
        ax.scatter(test_df_real[col1], test_df_real[col2], alpha=0.5, s=30,␣
 ↪label='Real', color='blue')
        ax.scatter(syn_df[col1], syn_df[col2], alpha=0.5, s=30,␣
 ↪label='Synthetic', color='orange')

        ax.set_xlabel(col1, fontsize=9)
        ax.set_ylabel(col2, fontsize=9)
        ax.set_title(f'{col1} vs {col2}', fontweight='bold', fontsize=10)
        ax.legend()
        ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('scatter_relationships.png', dpi=300, bbox_inches='tight')
print("Saved: scatter_relationships.png")
plt.show()

# Box plots

fig, axes = plt.subplots(2, 2, figsize=(10, 6))
axes = axes.flatten()

for idx, col in enumerate(continuous_cols[:4]):
    ax = axes[idx]

    data_to_plot = [test_df_real[col], syn_df[col]]

    bp = ax.boxplot(data_to_plot, labels=['Real', 'Synthetic'],␣
 ↪patch_artist=True)

    bp['boxes'][0].set_facecolor('lightblue')
    bp['boxes'][1].set_facecolor('lightyellow')

    ax.set_title(f'{col}', fontweight='bold')
    ax.set_ylabel('Value')
    ax.grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('boxplots_comparison.png', dpi=300, bbox_inches='tight')
print("Saved: boxplots_comparison.png")
plt.show()

#correlation

fig, axes = plt.subplots(1, 2, figsize=(8, 4))

# Real correlations
```

```
real_corr = test_df_real[continuous_cols[:15]].corr()
sns.heatmap(real_corr, ax=axes[0], cmap='coolwarm', center=0, vmin=-1, vmax=1,
            square=True, cbar_kws={'label': 'Correlation'})
axes[0].set_title('Real Data Correlations', fontsize=14, fontweight='bold')

# Synthetic correlations
syn_corr = syn_df[continuous_cols[:15]].corr()
sns.heatmap(syn_corr, ax=axes[1], cmap='coolwarm', center=0, vmin=-1, vmax=1,
            square=True, cbar_kws={'label': 'Correlation'})
axes[1].set_title('Synthetic Data Correlations', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('correlation_heatmaps.png', dpi=300, bbox_inches='tight')
print("Saved: correlation_heatmaps.png")
plt.show()

#QQ

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

for idx, col in enumerate(continuous_cols[:4]):
    ax = axes[idx]

    real_data = test_df_real[col].dropna()
    syn_data = syn_df[col].dropna()

    # Get quantiles
    real_quantiles = np.percentile(real_data, np.linspace(0, 100, 50))
    syn_quantiles = np.percentile(syn_data, np.linspace(0, 100, 50))

    # Plot
    ax.scatter(real_quantiles, syn_quantiles, alpha=0.6, s=50)

    # Diagonal line (perfect match)
    lims = [
        np.min([ax.get_xlim(), ax.get_ylim()]),
        np.max([ax.get_xlim(), ax.get_ylim()]),
    ]
    ax.plot(lims, lims, 'r--', alpha=0.75, linewidth=2, zorder=0)

    ax.set_xlabel('Real Data Quantiles')
    ax.set_ylabel('Synthetic Data Quantiles')
    ax.set_title(f'{col} - Q-Q Plot', fontweight='bold')
    ax.grid(alpha=0.3)

plt.tight_layout()
```

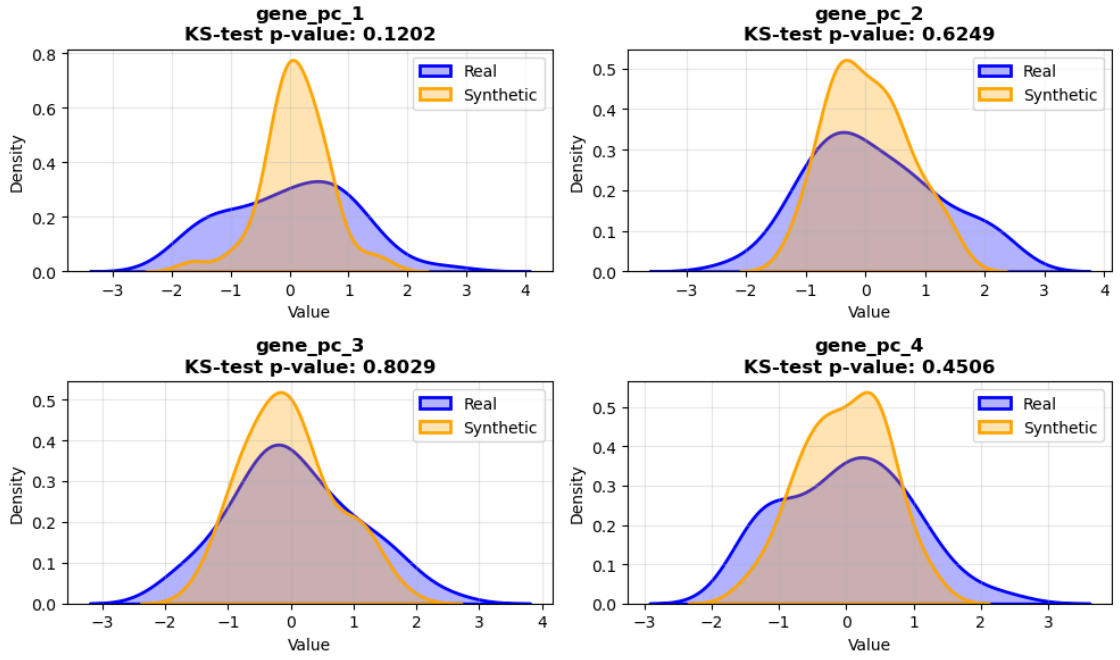
```
plt.savefig('qq_plots.png', dpi=300, bbox_inches='tight')
print("Saved: qq_plots.png")
plt.show()

#stats summary table

fig, ax = plt.subplots(figsize=(12, 6))
ax.axis('tight')
ax.axis('off')

stats_data = []
for col in continuous_cols[:10]:
    real_mean = test_df_real[col].mean()
    real_std = test_df_real[col].std()
    syn_mean = syn_df[col].mean()
    syn_std = syn_df[col].std()

    stat, pval = ks_2samp(test_df_real[col], syn_df[col])

    stats_data.append([
        col[:20],
        f'{real_mean:.4f}',
        f'{real_std:.4f}',
        f'{syn_mean:.4f}',
        f'{syn_std:.4f}',
        f'{pval:.4f}',
        ' ' if pval > 0.05 else ' '
    ])

columns = ['Feature', 'Real Mean', 'Real Std', 'Syn Mean', 'Syn Std', 'KS␣
 ↪p-value', 'Pass']
table = ax.table(cellText=stats_data, colLabels=columns, cellLoc='center',␣
 ↪loc='center',
                 colWidths=[0.2, 0.12, 0.12, 0.12, 0.12, 0.12, 0.08])

table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 2)

#dashboard
for i in range(len(columns)):
    table[(0, i)].set_facecolor('#4CAF50')
    table[(0, i)].set_text_props(weight='bold', color='white')

plt.title('Statistical Comparison - Real vs Synthetic Data', fontsize=14,␣
 ↪fontweight='bold', pad=20)
plt.savefig('statistics_table.png', dpi=300, bbox_inches='tight')
```

```
print("Saved: statistics_table.png")
plt.show()
```

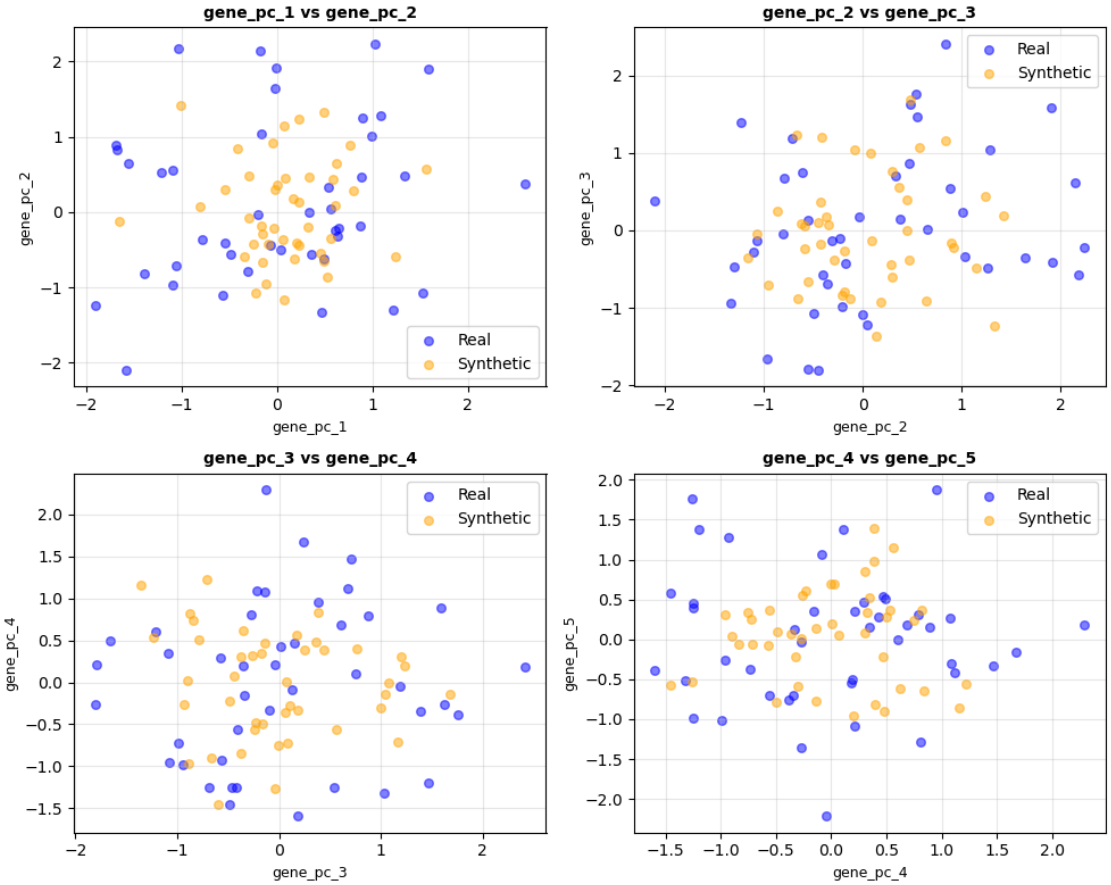
```
Real data shape: (43, 51)
Synthetic data shape: (43, 51)
Continuous columns: 50
  Saved: comparison_distributions.png
```



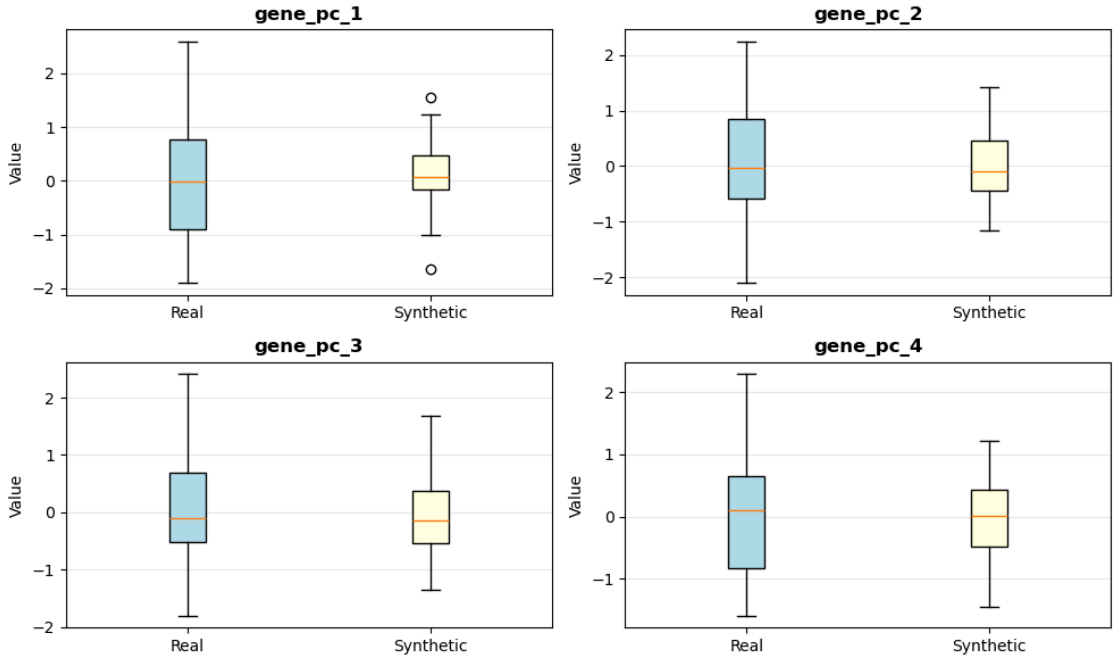

```
Saved: kde_distributions.png
```



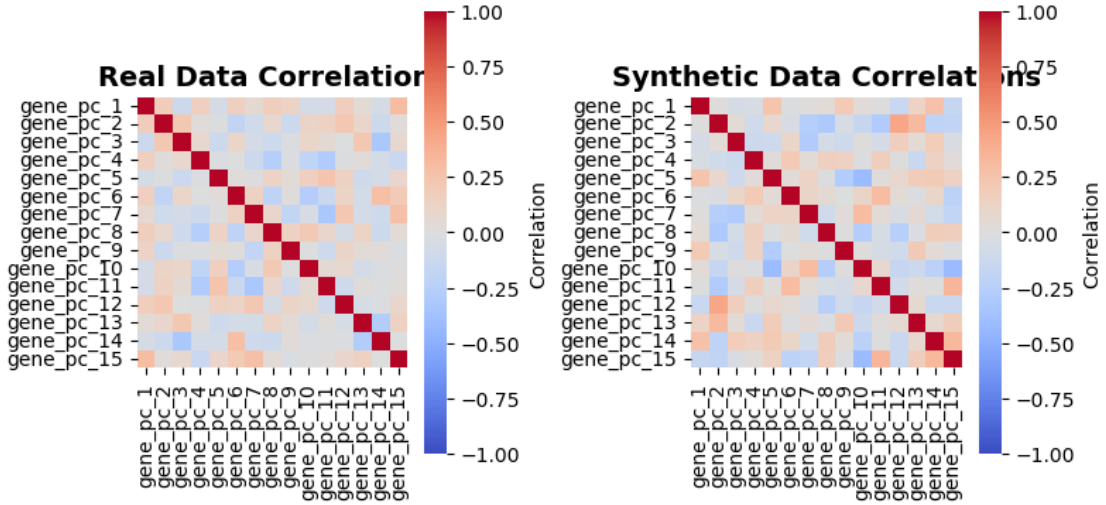

```
Saved: scatter_relationships.png
```



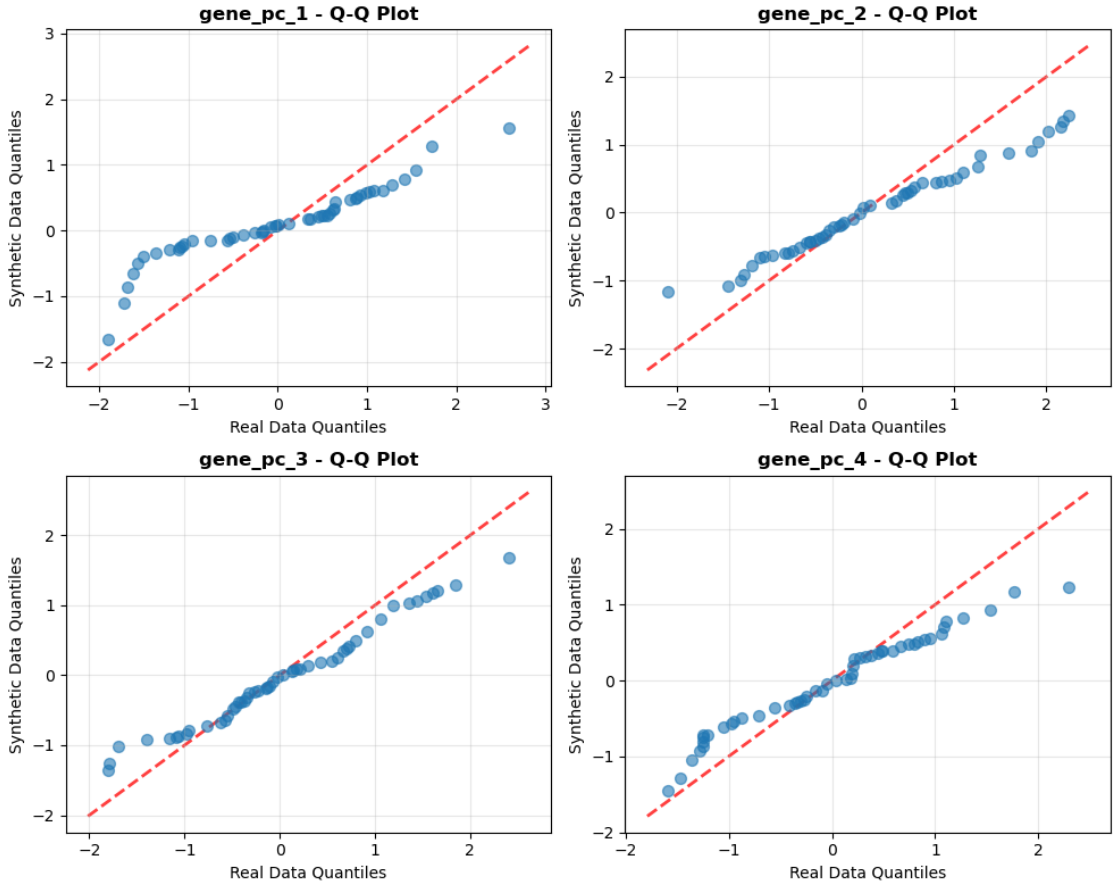
Saved: boxplots_comparison.png



Saved: correlation_heatmaps.png



Saved: qq_plots.png



Saved: statistics_table.png

Statistical Comparison - Real vs Synthetic Data

Feature	Real Mean	Real Std	Syn Mean	Syn Std	KS p-value	Pass
gene_pc_1	0.0023	1.0539	0.1009	0.5564	0.1202	✓
gene_pc_2	0.1381	1.0742	0.0311	0.6654	0.6249	✓
gene_pc_3	0.0453	0.9877	-0.0287	0.7271	0.8029	✓
gene_pc_4	-0.0054	0.9373	-0.0244	0.6313	0.4506	✓
gene_pc_5	0.0017	0.8448	0.0401	0.5842	0.8029	✓
gene_pc_6	0.0112	0.9247	-0.0291	0.6499	0.3056	✓
gene_pc_7	0.1332	1.0600	0.0050	0.6631	0.3056	✓
gene_pc_8	-0.0466	0.8102	-0.0341	0.5394	0.6249	✓
gene_pc_9	-0.0644	1.1499	0.1988	0.6054	0.0699	✓
gene_pc_10	-0.1940	0.6549	-0.0001	0.6351	0.4506	✓

1.4.1 Visual and statistical comparisons:

Multiple evaluation views were chosen to asses the synthetic data as it aligns to real data:

- 1. **KDE Curves** confirm smooth overlapping densities
- 2. **Box plots and Histograms** show comparable ranges and distribution shapes.
- 3. **Correlation heatmaps** indicate the diffusion model preserves the overal dependency on structure among the top principal components.
- 4. **Quantile- Quantile Plots** futher demonstrate that synthetic quantiles track real quantiles across most of value ranges, showing the strengh of the underlying structure.

The synthetic latent gene features closely mirror the real data across the first ten principal components.

Mean and standard deviation values remain well-aligned between real and synthetic distributions, and all **KS p-values exceed 0.05**, indicating no statistically significant drift.

Collectively this demonstrates that ***the diffusion model successfully captures the marginal structure of the underlying genetic latent space while preserving variability and avoiding mode collapse.***

1.5 Known Limitations and Possible Future Improvements

Although the current diffusion based SDGE produces realistic and diverse data, there are several limitations:

- 1. **Trade off with latent-only modeling**: Model poperates on PCA- reduced latent space rather than full expression, which may omit finer grain signals.
- 2. **Better clinical data: Better patient data**. A lot of clinical data was dropped or imputed.
- 3. **Small sample size**: TCGA-OV has usable sample size of 429 before preprocessing. This may limit the richness of the learned distribution. Some synthetic augmentation could help with this.

1.5.1 What I would do with more time:

- 1. Atchitecture Enhancements:
  - Experiment with learned encoder instead of PCA to capture nonlinear structure.
  - Compare and contrast different architecture solutions (tabDDPM vs CTGAN)
- 2. Deployment and Production
  - Build FASTAPI endpoint or develop lightweight UI Dsashboard (streanlit) where users can synthesize cohorts, visualize distributions and validate outputs.